**TESTE DE PRÁTICO LUMI – DESENVOLVEDOR(A) FULL STACK PLENO**

Olá, candidato(a)! Estamos empolgados em ter você no processo de seleção da Lumi. Como parte crucial deste processo, apresentamos um desafio técnico para avaliar suas habilidades.

Embora a vaga seja para a atuação como Desenvolvedor(a) Full Stack, **este desafio avaliará exclusivamente as suas habilidades de Backend e integração com Inteligências Artificiais.**

**Nota Importante:** O trabalho submetido para este teste não será utilizado para fins comerciais ou integrado em nossos produtos. O objetivo é puramente avaliativo.

## DESCRIÇÃO DO DESAFIO

Nós iremos fornecer algumas faturas de energia elétrica em formato PDF. Seu objetivo será desenvolver uma API RESTful capaz de:

- Receber e processar esses PDFs.
- Utilizar um **LLM (Large Language Model)**para interpretar e extrair os dados estruturados dessas faturas via análise multimodal (documentos/visão).
- Organizar e persistir esses dados em um banco de dados.
- Disponibilizar rotas (endpoints) para consulta e agregação desses dados.

**Tecnologias a serem utilizadas:** Typescript, Node.js (NestJS, Express etc. à sua escolha), banco de dados e integração com API de LLM (OpenAI, Gemini, Claude, etc., à sua escolha).

## DETALHAMENTO DO DESAFIO

1. **Extração de Dados via LLM:** Você deve desenvolver um extrator inteligente. O fluxo esperado é:

1.1. Upload e Análise Direta: Em vez de usar bibliotecas de extração de texto, seu código deve enviar o próprio arquivo PDF (fornecido no anexo FATURAS) diretamente para a API de um LLM com suporte a documentos/visão (como Gemini, OpenAI, Claude, etc.).

1.2. Engenharia de Prompt: Junto com o arquivo, você deve enviar um *prompt* bem estruturado, solicitando que o modelo analise o documento e retorne um objeto JSON estrito (JSON Mode/Structured Output) contendo exatamente as seguintes informações:

- **"Nº DO CLIENTE" (Ex: 7202210726)**

- "Mês de referência" (Ex: SET/2024)

Referente a
**SET/2024**

- 'Energia Elétrica' - Quantidade (kWh) e Valor (R$)
- 'Energia SCEEE s/ICMS' - Quantidade (kWh) e Valor (R$)
- 'Energia compensada GD I' - Quantidade (kWh) e Valor (R$)

| Itens da Fatura | Unid. | Quant. | Valores Faturados Preço Unit | Valor (R$) |
|---|---|---|---|---|
| Energia Elétrica | kWh | 100 | 1,04841351 | 104,81 |
| Energia SCEE s/ ICMS | kWh | 1.860 | 0,58125187 | 1.081,12 |
| Energia compensada GD I | kWh | 1.860 | 0,56148931 | -1.044,37 |

- 'Contrib Ilum Publica Municipal' - Valor (R$)

| | |
|---|---|
| Contrib Ilum Publica Municipal | 47,57 |

1.3. **Processamento e Variáveis Calculadas**

Após o LLM retornar o JSON estruturado, seu código Node.js deve calcular as seguintes variáveis de interesse antes de salvar os dados no banco:

- **Consumo de Energia Elétrica (kWh):** corresponde ao somatório das variáveis 'Energia Elétrica kWh' + 'Energia SCEEE s/ICMS kWh', por exemplo:
  - Exemplo (Abril/24): Consumo de Energia Elétrica = 50 kWh + 476 kWh = 526 kWh;
- **Energia Compensada (kWh):** corresponde à variável 'Energia Compensada GD I (kWh)';
- **Valor Total sem GD (R$):** somatório dos valores faturados de 'Energia Elétrica (R$)' + 'Energia SCEE s/ ICMS (R$)' + 'Contrib Ilum Publica Municipal (R$)';
- **Economia GD (R$):** corresponde à 'Energia compensada GD I (R$)'.

2. **Banco de Dados:** Implementar um banco de dados relacional (à sua escolha). A modelagem das tabelas deve ser pensada para suportar buscas com filtros e agregações de forma eficiente. Utilize um ORM de sua preferência (Prisma, TypeORM ou Sequelize).

3. **API RESTful:**

3.1. Implemente endpoints para expor os dados salvos. A API deve fornecer:

3.1.1. **Endpoint de Upload/Processamento:** Uma rota que receba o upload de uma fatura (PDF), faça a chamada ao LLM, calcule os valores e salve no banco de dados.

3.1.2. **Endpoint de Listagem (Biblioteca de Faturas):** Rota para listar as faturas já processadas, permitindo filtrar por numero_cliente e/ou período (mês de referência).

3.1.3. **Endpoints de Dashboard:** Rotas que retornem dados consolidados. Devem retornar:

- **Resultados de Energia (kWh):** Consumo de Energia Elétrica (kWh) vs Energia Compensada (kWh).

- **Resultados Financeiros (R$):** Valor Total sem GD (R$) vs Economia GD (R$).

4. **Testes Automatizados**

4.1. Escreva testes unitários para a sua aplicação. É essencial garantir o funcionamento isolado da lógica, portanto:

4.1.1. Crie *mocks* da resposta do LLM para não depender de chamadas externas de IA e consumo de tokens durante os testes.

4.1.2. Valide o cálculo correto das variáveis agregadas.

4.1.3. Valide o fluxo de rotas, a inserção no banco e o tratamento de erros (ex: caso o usuário envie um arquivo que não seja PDF ou o LLM falhe).

5. **Hospedagem:** A hospedagem da aplicação é opcional, mas pode ser um diferencial positivo. Você pode usar plataformas gratuitas como Vercel e Render para essa finalidade.

**O TESTE AVALIARÁ**

Este teste tem como objetivo avaliar suas habilidades técnicas, arquiteturais e de resolução de problemas no desenvolvimento de uma API robusta integrada à Inteligência Artificial. Serão considerados os seguintes aspectos:

1. **Integração com IA e Processamento de Dados:**
   a. **Engenharia de Prompt:** Capacidade de instruir o LLM de forma clara e restritiva para garantir o retorno dos dados em um formato estruturado (JSON Mode) e evitar alucinações.
   b. **Manipulação de Arquivos:** Eficiência no recebimento do upload (PDF) via API e no envio correto desse arquivo para o modelo multimodal.
   c. **Lógica de Negócio:** Precisão na conversão dos dados extraídos e robustez no cálculo das variáveis de interesse.
2. **Modelagem e Persistência de Dados:**
   a. **Design de Banco de Dados:** Adequação do esquema do banco de dados para armazenar os dados das faturas, permitindo fácil filtragem e agrupamento.
   b. **Eficiência e ORM:** Desempenho das *queries* para inserção e, principalmente, recuperação de dados consolidados para o Dashboard, utilizando adequadamente o ORM escolhido (Prisma/TypeORM/Sequelize).
3. **Desenvolvimento Backend (Node.js - NestJS/Express):**
   a. **Arquitetura:** Estrutura, separação de responsabilidades e organização do código.
   b. **Construção da API:** Implementação correta da API RESTful, seguindo boas práticas de rotas, verbos HTTP e *status codes*.
   c. **Resiliência e Segurança:** Tratamento rigoroso de erros e exceções, especialmente no upload de arquivos inválidos e no lidar com possíveis instabilidades/falhas da API do LLM.
4. **Qualidade do Código e Testes:**
   a. **Boas Práticas:** Legibilidade, clareza, modularização e aplicação de princípios de design de software (SOLID, Clean Code).

b. **Testabilidade:** Qualidade dos testes unitários implementados, capacidade de criar *mocks* eficientes para serviços externos (LLM) e garantia de cobertura da lógica principal.

**ENTREGA**

- **Código:** O código-fonte deve ser entregue preferencialmente através de um repositório **privado** no GitHub (ou enviado compactado em formato .zip). **Importante: Não deixe o seu repositório público.** Para que possamos avaliar o seu teste no GitHub, adicione a Tech Lead como colaboradora no repositório utilizando o username **elianemaciel** ou o e-mail **eliane.faveron@gmail.com**.
- **Documentação:** Fornecer um arquivo README.md detalhado com instruções claras de configuração, instalação, execução, exemplos de requisições para a API e quais variáveis de ambiente são necessárias (ex: chaves de API do LLM). É fundamental também documentar no README as suas decisões arquiteturais e tecnologias escolhidas (ex: qual framework Node.js, qual ORM e qual LLM foram utilizados e o porquê dessas escolhas).
- **Demonstração (Opcional):** Hospede a API em um serviço de sua escolha (como Vercel, Render, Railway, etc.) e nos envie o link base (URL) para que possamos testar os *endpoints* da sua aplicação em nuvem.

**Como enviar:** Envie a sua entrega para os e-mails: **gabriel@labs-lumi.com.br** e **eliane@labs-lumi.com.br**.

O e-mail deve conter:

1. O link do repositório Git (certifique-se de que o acesso já foi concedido) ou o arquivo de código compactado em anexo.
2. O link para o seu perfil no LinkedIn ou o seu currículo atualizado em formato PDF em anexo.
3. O link da API hospedada (caso tenha optado por fazer o deploy da demonstração).
4. Qualquer observação que julgar necessária sobre as decisões técnicas tomadas.

**Data limite para envio do teste prático: 01/03/2026 às 23:59, testes enviados após o prazo limite serão desconsiderados.**

Boa sorte! Estamos ansiosos para ver o que você irá criar!